Pipeline de Dados com IoT e Docker

Disruptive Architectures: IoT, Big Data e IA (UniFECAF)
Mikael Icaro Simões, RA 96278

1. Contextualização

Dispositivos IoT geram leituras o tempo todo: um sensor de temperatura pode registrar um valor a cada minuto, e em poucos meses isso vira dezenas de milhares de linhas. Guardar esse volume em planilha ou em arquivo solto funciona no começo, mas logo fica difícil responder perguntas simples, como qual foi a temperatura máxima de cada dia ou em que horário o sensor mais trabalha.

A proposta deste trabalho é montar um pipeline pequeno que resolve esse problema de ponta a ponta: ler os dados brutos de um sensor, gravar num banco de dados relacional, organizar as consultas em views SQL e mostrar o resultado num dashboard interativo. Tudo roda em container, para que qualquer pessoa consiga repetir o ambiente com poucos comandos.

Os dados vêm do dataset "Temperature Readings: IoT Devices", do Kaggle. São 97.606 leituras de temperatura de um sensor entre 28/07/2018 e 08/12/2018, com a indicação de cada leitura ser de ambiente interno ou externo.

2. Passos realizados

Ambiente. Criei um ambiente virtual Python e instalei pandas, SQLAlchemy, psycopg2, Streamlit e Plotly. O Git foi configurado com nome e e-mail e o projeto foi versionado desde o início.

Container do PostgreSQL. O banco sobe com `docker compose up -d`, a partir de um arquivo `docker-compose.yml`. Ele cria o container `postgres-iot` com a imagem oficial do PostgreSQL 16, expõe a porta 5432 apenas para a própria máquina e guarda os dados num volume, para que não se percam quando o container é desligado.

Carga dos dados. O script `src/carga.py` lê o CSV com pandas, renomeia as colunas (o nome original `room_id/id` é ruim para usar em SQL), converte a data do formato dia-mês-ano para timestamp e traduz `In` e `Out` para `interno` e `externo`. O arquivo tem um `id` repetido, então ficou só a primeira ocorrência: foram 97.605 linhas gravadas na tabela `temperature_readings`. A gravação é feita com SQLAlchemy.

Views SQL. Depois da carga o mesmo script cria as três views descritas na próxima seção, a partir do arquivo `sql/views.sql`.

Dashboard. O arquivo `src/dashboard.py` lê cada view com `pandas.read_sql` e monta um gráfico Plotly para cada uma, dentro de uma página Streamlit. Para ver o resultado, `streamlit`

`run src/dashboard.py`.

Conferência. Depois da carga conferi a contagem direto no banco (97.605 linhas, igual ao CSV menos a duplicada) e consultei cada view com `psql` antes de montar os gráficos.

3. As três views SQL

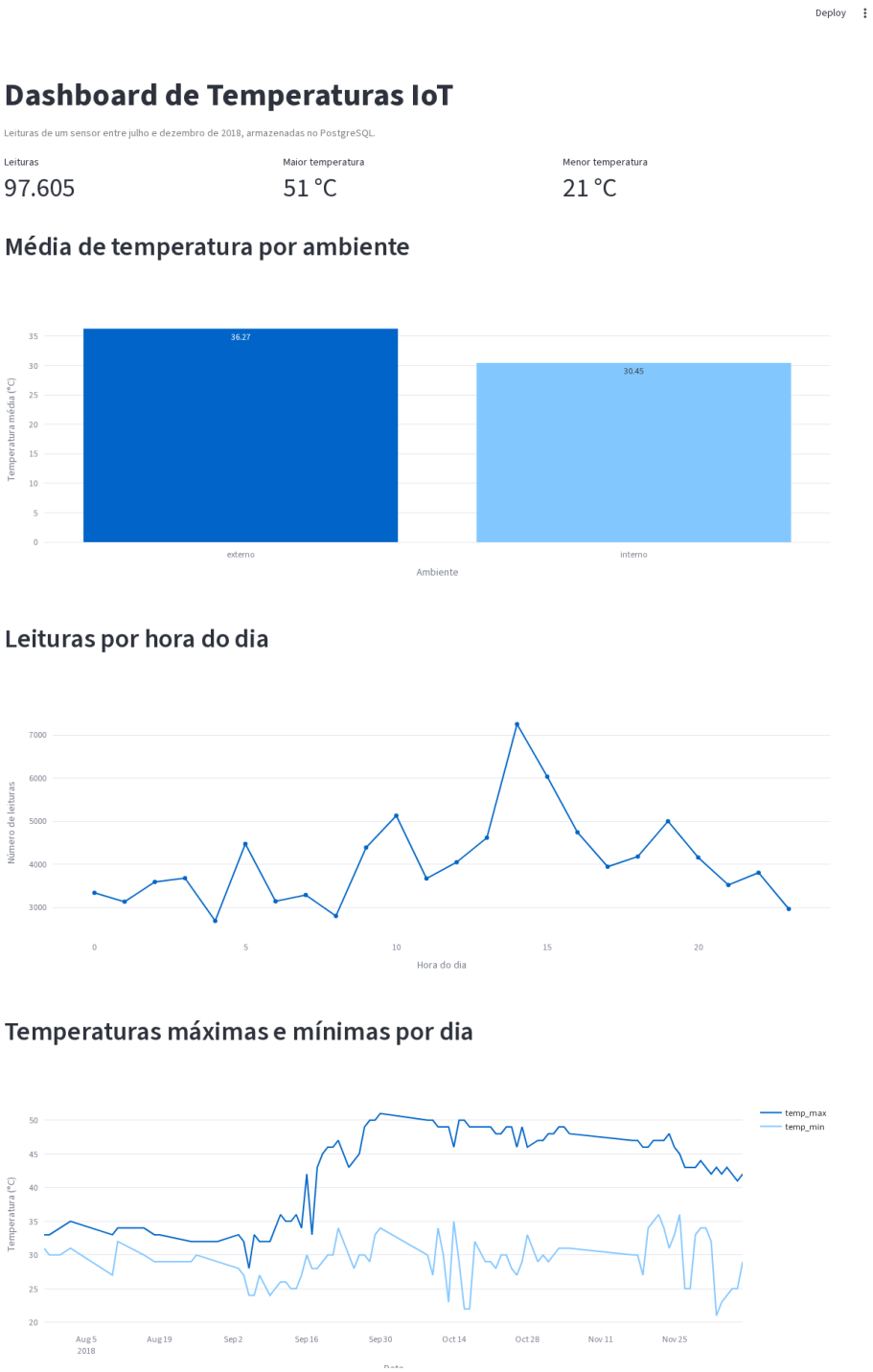
`avg_temp_por_dispositivo`. Calcula a temperatura média e o total de leituras por dispositivo e por ambiente. O dataset tem um único dispositivo, o "Room Admin", então agrupar só por dispositivo daria uma barra. Agrupar também por ambiente mostra a diferença que existe de fato: média de 30,45 °C no interno e 36,27 °C no externo.

`leituras_por_hora`. Conta quantas leituras caem em cada hora do dia, de 0 a 23. Responde quando o sensor está mais ativo e ajuda a perceber horários mal cobertos.

`temp_max_min_por_dia`. Devolve a temperatura máxima, a mínima e a média de cada dia. É a visão para acompanhar a evolução ao longo dos meses e achar dias fora do normal.

4. Visualizações

O dashboard tem três indicadores no topo (total de leituras, maior e menor temperatura) e três gráficos interativos, um para cada view: barras para a média por ambiente, linha para as leituras por hora e linha dupla para as temperaturas máxima e mínima por dia. Ao passar o mouse sobre os gráficos o Plotly mostra o valor exato de cada ponto, e dá para dar zoom em qualquer período.



5. Insights e uso prático

O que os dados mostram

- O ambiente externo varia bem mais que o interno. O desvio padrão é 5,7 °C no externo e 2,2 °C no interno, o que faz sentido: o interno é mais protegido.
- O horário com mais leituras é 14h, com 7.248, seguido de 15h, com 6.029. O menor volume é às 4h, com 2.690.
- A maior temperatura foi 51 °C, em 30/09/2018, e a menor foi 21 °C, em 03/12/2018.

- O volume de dados é muito desigual. Julho tem 154 leituras, agosto 2.241 e setembro 42.740. A máxima diária parece subir em setembro, mas parte disso é só o sensor ter começado a registrar muito mais, então não dá para afirmar que houve aquecimento real.
- Existem dias sem nenhuma leitura. A maior lacuna vai de 06/11 a 16/11, 11 dias seguidos.

Uso em um ambiente real

- Alerta de falha de coleta: a lacuna de novembro seria detectada sozinha se houvesse uma consulta que procura dias sem leitura, avisando a equipe de manutenção.
- Monitoramento de ambiente controlado, como câmara fria, sala de servidores ou estufa, com alerta quando a máxima diária passa de um limite.
- Planejamento de manutenção e consumo de energia, cruzando os horários de maior temperatura com o uso de ar-condicionado.
- Escala: com milhões de leituras, as views poderiam virar tabelas agregadas atualizadas por agendamento, e a ingestão poderia trocar o CSV por uma fila de mensagens, mantendo o banco e o dashboard como estão.

6. Conclusão

O pipeline cumpre o objetivo: dados brutos entram, ficam organizados num banco relacional e saem como informação visual. Usar container deixou o ambiente repetível, e as views separaram a regra de consulta do código de visualização, o que facilita trocar o gráfico sem mexer no SQL. A principal limitação encontrada está nos próprios dados, com um só dispositivo e cobertura irregular, e não na tecnologia.

Repositório: https://github.com/Mikael-Icaro/Disruptive-Architectures-unifecaf